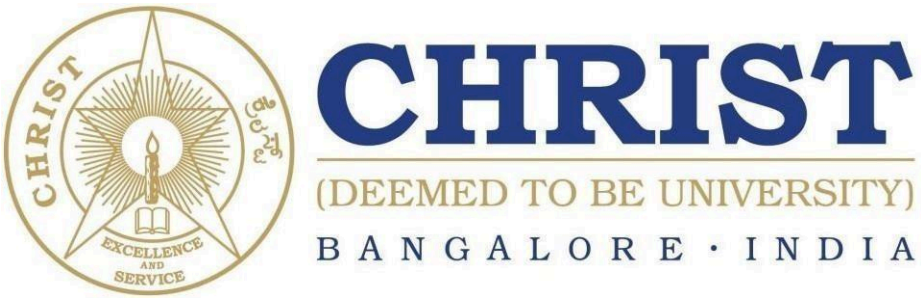


AI-powered Smart Agriculture Advisory System

by

Rakshitha A Manoj

(2241365)

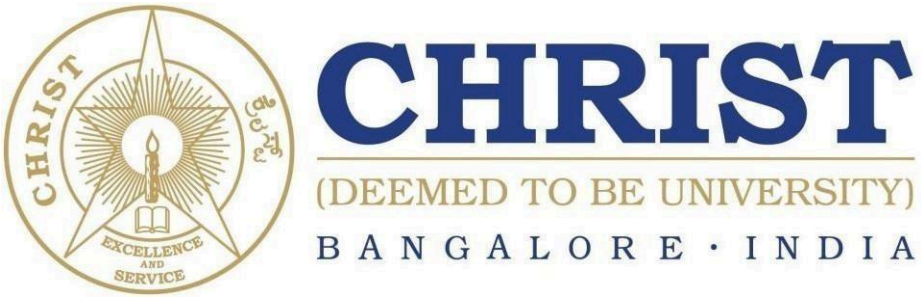
Under the guidance of

Dr. Mahalakshmi J

Assistant Professor

A Project report submitted in partial fulfillment of the requirements for the award of degree of Bachelor of Computer Applications of CHRIST (Deemed to be University)

March – 2025



CERTIFICATE

*This is to certify that the report titled **AI-powered Smart Agriculture Advisory System** is a bonafide record of work done by **Rakshitha A Manoj (2241365)** of CHRIST (Deemed to be University), Bengaluru, in partial fulfillment of the requirements of 6th Semester Bachelor of Computer Applications during the academic year 2024-25*

Head of the Department **Project Guide**

Valued-by

	Name	:	
1.	Register Number	:	
	Examination Center	:	CHRIST (Deemed-to-be) University
2.	Date of Exam	:	

ACKNOWLEDGEMENT

First of all, I thank the Lord almighty for his immense grace and blessings showered on me at every stage of this work.

I profoundly thank Dr. Fr Benny Thomas, Director, and Dr. Fr Biju K Chacko, Associate Director, CHRIST (Deemed to be University) - Bangalore Yeshwanthpur Campus, for providing me the opportunity to be a part of the institution.

I am greatly indebted to Dr. Vinay M, Head of the Department, Computer Science, CHRIST (Deemed to be University) - Bangalore Yeshwanthpur Campus for providing the opportunity to take up this project as part of my curriculum.

I am greatly indebted to our Project Coordinator, Dr. Sindhu V, for providing the opportunity to take up this project and for helping with his valuable suggestions.

I am deeply indebted to my project guide, Dr. Mahalakshmi J, for her assistance and valuable suggestions as a guide. Her critical suggestions helped me to improve the project work.

I express my sincere thanks to all Faculties of the Department of Computer Science, CHRIST(Deemed to be University), for their valuable suggestions during the course of this project.

Acknowledging the efforts of everyone, their chivalrous help in the course of the project preparation and their willingness to corroborate with the work, their magnanimity through lucid technical details led to the successful completion of my project.

I would like to express my sincere thanks to all my friends, colleagues, parents and all those who have directly or indirectly assisted me during this work.

Rakshitha A Manoj

ABSTRACT

Agriculture is constantly plagued by crop disease, leading to heavy yield loss and financial losses. Conventional disease detection involves manual inspection, which is time-consuming, unreliable, and usually inaccessible to small-scale farmers. AgriVision is a computer vision-based smart agriculture advisory system incorporating AI-based crop disease detection, severity analysis, real-time weather information, and a chatbot for farm support. It uses MASE-Net (Multi-Stage Attention & Severity Estimation Network), a CNN-Transformer hybrid model, for severity estimation and disease classification. It processes images through MobileNetV3 for extracting features, followed by MobileViT for greater contextual awareness. The AI model is quantized and pruned to optimize so that it supports quick ONNX-based mobile inference for real-time and offline usage. The system includes a FastAPI backend alongside MongoDB for storage and real-time analytics. Offline storage using SQLite allows users to see reports generated earlier in low-connectivity areas. AgriVision also contains an AI chatbot with FAISS-based retrieval using Sentence-BERT and TinyBERT for response generation, which provides farmers recommendations for disease management, treatment, and optimal agriculture practices. The detection process involves image preprocessing, feature extraction, classification, and severity estimation based on affected leaf area. Detection metadata like timestamp, crop type, disease class, and severity percentage are saved securely for reporting. User-friendly, AgriVision is made accessible to technologically low-skilled farmers, allowing them to make smart decisions and minimize crop loss. Powered by the strength of computer vision, NLP, predictive analytics, and mobile AI inference, AgriVision provides a scalable, intelligent, and real-time solution to precision agriculture and effective disease management.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

1. INTRODUCTION

Agriculture is a vital sector that sustains global food production and economic stability, yet it continues to face major challenges such as crop diseases, climate unpredictability, and inefficient farming methods. Among these, plant diseases significantly impact crop yields, leading to economic losses for farmers and disruptions in food supply chains. Traditional disease detection relies on manual inspection, which is often inaccurate, slow, and dependent on expert knowledge. Delayed or incorrect diagnoses contribute to disease spread, excessive pesticide use, and reduced farm productivity.

To address these challenges, AI-driven solutions are becoming increasingly essential. AgriVision is an AI-powered smart agriculture advisory system that integrates AI-driven crop disease detection, severity estimation, real-time weather insights, and an intelligent chatbot for agricultural support. Using a CNN-Transformer hybrid model (MASE-Net), AgriVision processes plant images to classify diseases with high accuracy and estimate infection severity. The system also includes an AI-driven chatbot utilizing FAISS-based retrieval with Sentence-BERT and TinyBERT, providing disease management recommendations, treatment strategies, and best farming practices. Unlike traditional solutions that require constant internet connectivity, AgriVision is optimized for ONNX/TFLite-based on-device AI inference, ensuring real-time functionality even in remote areas with limited connectivity.

The primary objective of AgriVision is to enhance agricultural productivity by providing farmers with an intelligent, mobile-based decision-support system. It aims to reduce crop losses, optimize pesticide use, and deliver fast, reliable disease diagnosis without requiring specialized expertise.

The problem statement guiding this project is that existing disease detection methods are slow, inefficient, and often inaccessible to many farmers, leading to delayed intervention and reduced yields. There is a critical need for an AI-powered mobile solution that can instantly diagnose diseases, estimate severity, and provide actionable insights. AgriVision addresses these issues by deploying lightweight AI models that enable real-time disease detection and guidance, even in low-connectivity environments.

1.1 OVERVIEW OF THE SYSTEM

The AI model, MASE-Net (Multi-Stage Attention & Severity Estimation Network),

integrates MobileNetV3 for feature extraction, MobileViT for attention-based disease identification, and a severity estimation module that analyzes the affected leaf area to determine disease severity (0-100%). Optimized for ONNX Runtime, MASE-Net enables real-time, offline disease classification and severity estimation on mobile devices without requiring an internet connection. Additionally, the system integrates weather insights, providing farmers with environmental data to support better farming decisions and disease prevention strategies.

The system is built with a Flutter-based frontend, delivering an interactive and visually engaging user experience. The FastAPI backend manages AI inference, user authentication, and database operations. MongoDB is used to store disease reports, user history, and chatbot interactions. The AI model, MASE-Net (Multi-Stage Attention & Severity Estimation Network), integrates MobileNetV3 for feature extraction, MobileViT for attention-based disease identification, and a severity estimation module that analyzes the affected leaf area to determine disease severity (0-100%). Optimized for ONNX Runtime, MASE-Net enables real-time, offline disease classification and severity estimation on mobile devices without requiring an internet connection. In addition to disease detection, AgriVision features an AI-powered chatbot utilizing FAISS-based retrieval with Sentence-BERT and TinyBERT, offering treatment recommendations, pesticide usage guidance, and general agricultural advice. This reduces farmers' reliance on expert consultations and makes advanced agricultural knowledge more accessible. The system also supports disease history tracking and report storage, allowing farmers to review past detections and make data-driven decisions over time.

By integrating AI-driven disease detection, severity estimation, chatbot assistance, and weather insights into a single mobile application, AgriVision provides a comprehensive solution for modern precision farming. Its ability to function offline, offer real-time analysis, and provide actionable insights makes it an effective tool for improving crop health management and increasing agricultural efficiency.

1.2 RESEARCH CONTRIBUTION

As part of advancements in AI-driven precision agriculture, prior research efforts led to the development of a novel deep learning model, MSACN (Multi-Scale Adaptive Convolutional Network), designed for crop disease detection. MSACN incorporates adaptive multi-scale convolutional layers to improve feature extraction, significantly enhancing disease

classification accuracy compared to traditional CNN models like ResNet, InceptionV3, and Xception. Tested on a dataset of 25,000+ images spanning five major crop types (Rice, Wheat, Corn, Tomato, and Strawberry), MSACN achieved a 98.64% accuracy, outperforming existing architectures.

This research was accepted at the Annual International Congress on Computer Science (AICCS) 2025, scheduled for April 17-18, 2025, in Oxford, United Kingdom. Additionally, it is set to be published in the Proceedings on Engineering Sciences (PES) Journal, which is indexed in SCOPUS and Google Scholar, ensuring global academic recognition.

While MSACN achieved high accuracy, its computational complexity made it unsuitable for real-time mobile inference. This led to the development of MASE-Net (Multi-Stage Attention & Severity Estimation Network), a lightweight CNN-Transformer hybrid optimized for mobile deployment in AgriVision. MASE-Net ensures efficient disease classification and severity estimation, aligning with the project's goal of providing real-time, on-device disease detection for farmers.

Furthering this research, the MASE-Net model was presented and published at the International Conference on Recent Trends in Computational Technologies and Sustainable Development Goals (ICRTCTS - 2025), organized by PSG College of Arts and Science. The paper is included in the conference proceedings under ISBN 978-81-986118-7-1, contributing to the broader academic discourse on AI-driven precision agriculture and sustainable farming practices.

Additionally, a theoretical study was conducted on Crop Disease Detection in Diverse Environments Using Generative Adversarial Networks (GANs). This research explored GAN-based data augmentation techniques to improve model generalization across varying environmental conditions, ensuring robust disease classification in real-world settings. While no implementation was carried out, the study provided valuable insights into the potential of GANs for mitigating dataset variability in agricultural applications, laying the groundwork for future experimental validation.

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

To assist farmers, several mobile applications have been developed for disease detection and agricultural advisory services. Plantix is one of the most widely used apps, allowing farmers to upload images of affected plants for diagnosis. However, it requires an active internet connection and relies on a pre-existing disease database, which may not cover all crops or region-specific plant diseases. Another popular application, AgroAI, provides AI-powered crop disease detection but has limited classification capabilities and does not estimate infection severity. FarmRise offers weather alerts, soil analysis, and farming tips, but lacks real-time AI-based disease detection. Similarly, Leaf Doctor and CropDiagnosis analyze plant health from leaf images, but these tools are primarily research-focused rather than practical solutions for farmers.

While these applications represent significant progress in digital agriculture, they still have critical drawbacks. Most require internet connectivity to function, making them unreliable for farmers in rural areas with poor network access. Additionally, they lack real-time AI inference on mobile devices, often depending on cloud-based servers for image processing, leading to delays in obtaining results. Many existing apps also focus on a limited set of crops, making them less effective for diverse agricultural regions where farmers cultivate multiple crop varieties.

2.1.1 Limitations of Existing System

Despite advancements in mobile-based crop disease detection, existing systems struggle to provide real-time, offline, and highly accurate diagnoses. One of the biggest limitations is subjectivity in manual disease identification, as farmers rely on experience and visual observation, leading to misdiagnosis and delays in intervention. Even when using mobile applications, farmers often face internet dependency issues, making it difficult to access timely disease assessments in areas with low or unstable network coverage.

Another major drawback is the limited crop and disease coverage in existing apps. Many AI-based solutions focus on common diseases for major crops, leaving out region-specific plant infections that require localized datasets. Additionally, most mobile apps do not estimate disease severity, which is crucial for farmers to determine the extent of infection and make informed treatment decisions. This limitation forces farmers to rely on

trial-and-error methods when applying pesticides, potentially leading to overuse of chemicals, environmental damage, and increased costs.

Furthermore, laboratory-based disease detection, though highly accurate, is expensive, time-consuming, and impractical for most small and medium-scale farmers. Advanced remote sensing solutions, such as drones and satellite-based disease monitoring, are primarily used in large-scale commercial farming due to their high costs and technical complexity. These challenges highlight the urgent need for an affordable, AI-powered, offline disease detection system that can provide real-time, region-specific crop health insights without requiring internet access or expert intervention. AgriVision addresses these issues by integrating on-device AI inference with ONNX Runtime, enabling accurate, fast, and offline disease detection and severity estimation directly on mobile devices.

2.2 PROPOSED SYSTEM

To overcome the limitations of existing crop disease detection methods, AgriVision is designed as a real-time, AI-powered mobile application that provides automated disease classification and severity estimation directly from plant images. Unlike traditional approaches that rely on manual inspection, expert consultations, or cloud-based processing, AgriVision utilizes on-device AI inference, enabling instant, offline disease detection without requiring an internet connection.

The system is built using MASE-Net (Multi-Stage Attention & Severity Estimation Network), which combines MobileNetV3 for feature extraction, MobileViT for attention-based disease identification, and an attention-driven severity estimation module that analyzes the affected leaf area to determine disease severity (0-100%). This severity assessment helps farmers determine the extent of infection and take timely, informed action. AgriVision also integrates an AI-powered chatbot utilizing FAISS-based retrieval with Sentence-BERT and TinyBERT, providing personalized treatment recommendations, pesticide usage guidance, and best farming practices—ensuring that farmers receive expert-level assistance without needing external consultations. Additionally, weather insights are incorporated via a Weather API, allowing farmers to consider temperature, humidity, and rainfall data when planning disease prevention and farm management strategies.

To ensure a seamless user experience, AgriVision features a Flutter-based frontend with an intuitive interface, allowing farmers to capture or upload images, view results instantly, and

receive expert recommendations. The FastAPI backend manages API requests, data storage, and AI inference processing, ensuring secure and efficient disease detection. The system uses MongoDB for storing disease reports, chatbot interactions, and historical detections, ensuring efficient data retrieval and personalized assistance. Unlike existing applications that depend on internet connectivity and cloud computing, AgriVision is optimized for ONNX Runtime, allowing it to function independently on mobile devices. This makes the system highly reliable for farmers in remote areas with limited access to high-speed internet.

2.2.1 Benefits of Proposed System

AgriVision eliminates internet dependency by leveraging on-device AI inference, enabling real-time, offline disease detection even in remote areas. Unlike cloud-based solutions that require stable connectivity, AgriVision processes images instantly on mobile devices using ONNX Runtime, ensuring fast, reliable results without delays. This makes it highly suitable for farmers with limited or no internet access, allowing them to diagnose crop diseases without relying on external servers.

A key advantage of AgriVision is its disease severity estimation, which categorizes infections as Low, Moderate, or High, helping farmers gauge the extent of crop damage. Unlike conventional disease detection apps that merely classify diseases, AgriVision provides actionable insights, enabling farmers to determine urgency and apply treatments accordingly. By offering severity-based recommendations, the system optimizes pesticide use, preventing excessive chemical application and reducing farming costs while promoting sustainable agricultural practices.

In addition to disease detection, AgriVision features an AI-powered chatbot utilizing FAISS-based retrieval with Sentence-BERT and TinyBERT, offering personalized treatment strategies, pesticide usage guidance, and best farming practices. The system also integrates weather insights, helping farmers make informed decisions about irrigation and disease prevention based on temperature, humidity, and rainfall data. Unlike expensive laboratory tests or drone-based monitoring, AgriVision provides a cost-effective, scalable alternative, making AI-driven disease detection accessible to both small and large-scale farmers. By combining real-time AI analysis, offline functionality, and expert guidance, AgriVision empowers farmers with precise, data-driven decisions to improve crop health and yield.

2.3 LITERATURE REVIEW

Crop diseases significantly impact agricultural productivity, leading to economic losses and food insecurity. Traditional methods for disease detection rely on manual inspection, which is often slow, labor-intensive, and requires expert knowledge. Recent advancements in deep learning and computer vision have provided effective solutions for automating crop disease detection and severity estimation. Convolutional Neural Networks (CNNs) have shown remarkable success in plant disease classification, demonstrating high accuracy across various datasets. Studies have extensively applied CNN models to classify plant diseases in crops such as wheat, maize, and tomatoes. Research conducted by Mohanty et al. utilized a deep CNN model trained on the PlantVillage dataset, achieving an accuracy exceeding 99% in classifying multiple plant diseases [11]. Similarly, Kulkarni et al. developed an image-processing pipeline integrated with machine learning for plant disease detection, enhancing classification accuracy through preprocessing and feature extraction techniques [1].

Further studies have demonstrated the effectiveness of various CNN architectures in plant disease classification. Albahli and Masood introduced an attention-based CNN model (EANet) for multi-class maize disease classification, significantly improving accuracy by capturing fine-grained features [15]. Kundu et al. focused on integrating CNNs with domain adaptation techniques, enabling improved performance for disease detection across different environments [17]. In addition, Roy et al. proposed a fine-grained object detection model using YOLOv4, achieving high accuracy in real-time plant disease classification [2]. These studies emphasize the effectiveness of deep learning in automating disease diagnosis, but challenges remain in optimizing models for real-world deployment on mobile and edge computing platforms.

Estimating disease severity is another critical aspect of crop health monitoring. Traditional methods rely on subjective human assessment, which often results in inconsistencies. Recent research has focused on developing automated deep learning models to quantify the severity of crop diseases based on image analysis. Wang et al. developed a deep learning-based severity estimation model that uses pixel-wise segmentation to determine the affected leaf area [16]. Bock et al. emphasized the importance of transitioning from manual severity scoring to sensor-based automated measurements, which enhance accuracy and consistency [6]. Additionally, Faye et al. conducted a survey on machine learning-based severity estimation methods, highlighting the role of multi-scale feature extraction in

improving severity quantification [18]. Similarly, Kuska et al. explored early disease resistance detection in barley using hyperspectral imaging, demonstrating that deep learning models can identify disease severity in early infection stages [10]. These studies indicate that deep learning can be effectively used not only for disease classification but also for assessing the extent of infection.

The development of hybrid models that integrate CNNs and Vision Transformers (ViTs) has gained attention in recent years. While CNNs are efficient in feature extraction, ViTs improve classification by capturing long-range dependencies within images. Mehta and Rastegari introduced MobileViT, a lightweight vision transformer optimized for mobile-based applications, demonstrating superior performance over traditional CNNs [13]. Zhu et al. combined convolutional networks with transformers to enhance accuracy in crop disease identification, proving that hybrid models outperform standalone CNN architectures in feature learning [22]. Wang et al. implemented an attention-based neural network with Bayesian optimization for rice disease classification, showing that transformer-based models can significantly improve detection accuracy [14]. These findings suggest that integrating transformers into deep learning architectures can enhance disease classification while maintaining computational efficiency, making them suitable for mobile and edge computing applications.

With the increasing need for real-time disease detection, mobile and edge computing solutions have become a research focus. Deploying deep learning models on mobile devices allows farmers to diagnose diseases instantly without requiring cloud-based processing. El Jarroudi et al. explored the use of edge artificial intelligence in sustainable agriculture, emphasizing the importance of lightweight AI models for on-device disease detection [4]. Saleem et al. developed a mobile-based plant disease detection system that balances model size and inference speed, ensuring efficient real-time classification [21]. Tembhurne et al. demonstrated that lightweight CNNs could be deployed on mobile applications for accurate disease detection without requiring extensive computational resources [12]. In addition, Ataman and Eroglu performed a comparative analysis of different CNN architectures, confirming that MobileNet-based models are well-suited for real-time plant disease classification on edge devices [24]. These studies highlight the feasibility of deploying disease detection models on mobile platforms, aligning with the objectives of AgriVision, which integrates optimized CNN-Transformer architectures with mobile AI capabilities.

Despite the advancements in deep learning-based crop disease detection, several challenges

remain. One of the primary challenges is the lack of high-quality annotated datasets for training deep learning models. Many studies rely on publicly available datasets such as PlantVillage, which may not represent real-world variations in lighting, angles, and disease symptoms. Furthermore, the absence of standardized severity labels affects the generalizability of severity estimation models [5]. Another challenge is ensuring model interpretability, as deep learning models are often criticized for being "black boxes." Schramowski et al. proposed methods for enhancing the interpretability of neural networks by interacting with model explanations, an essential aspect of building trust in AI-driven agricultural solutions [7]. Additionally, researchers have explored transformer-based attention mechanisms to improve the explainability of disease classification and severity estimation models [22].

Another limitation is optimizing deep learning models for mobile deployment. While MobileNetV3 and MobileViT offer lightweight alternatives, further research is required to implement quantization and pruning techniques that enhance inference speed without compromising accuracy. Bock et al. emphasized the need for efficient sensor-based disease monitoring systems, integrating AI with hardware optimizations to enable real-time decision-making in the field [6]. Studies by Wang et al. and Kuska et al. indicate that combining hyperspectral imaging with deep learning could enhance detection accuracy, but integrating such advanced imaging technologies into mobile applications remains a challenge [10][16].

2.4 FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS

Table 2.1 Functional Requirements

Requirement ID	Requirement Name	Description
A_FR1	User Authentication	The system should provide secure login access, allowing users to access features like disease detection, severity estimation, and AI-based recommendations.
A_FR2	Dashboard Display	The dashboard should present an overview of recent disease detection results, severity levels

		(Low, Moderate, High), and past analysis history for quick access.
A_FR3	Crop Health Data Storage	The system should allow users to store and retrieve previous disease detection results, severity levels, and AI-generated recommendations for future reference.
A_FR4	Image-Based Disease Detection	Farmers should be able to capture or upload plant images, which the system will process using AI models to classify diseases and estimate severity levels.
A_FR5	Disease Management Assistance	The AI-powered chatbot should provide context-aware recommendations on treatment strategies, pest control, and preventive measures based on disease severity and type.
A_FR6	Search & History	Users should be able to search and filter past disease detection records and retrieve previous AI recommendations for reference.
A_FR7	Offline Mode Support	The system should allow offline storage of disease reports and enable users to access past records and recommendations without an internet connection.
A_FR8	Weather API Integration	The system should provide weather data (humidity, temperature, rainfall) to help farmers make informed decisions on disease prevention and farm management.

Table 2.2 Non-Functional Requirements

Requirement ID	Requirement Name	Description
A_NF_R1	Performance	The AI models should be optimized using ONNX Runtime for fast inference. Image processing should be efficient to reduce lag, enabling real-time disease analysis. The UI should be lightweight, with minimal animations, ensuring smooth performance even on low-end devices with at least 3GB RAM and a quad-core processor.
A_NF_R2	Safety & Security	User data should be encrypted to ensure privacy, and password-based authentication should be implemented to prevent unauthorized access. The system should periodically back up detection records locally to prevent data loss.
A_NF_R3	Usability	The application should be designed for farmers with varying technical expertise, ensuring a simple, intuitive, and easy-to-navigate UI. It should include user guides, tutorials, and multilingual support for accessibility. The UI should be optimized for mobile devices, featuring clear layouts and minimal complexity.
A_NF_R4	Reliability	The system should ensure consistent AI-driven disease detection accuracy and include robust error handling. Data validation mechanisms should be in place to prevent incomplete or incorrect disease reports. Local storage (SQLite) and MongoDB backend storage

		should ensure data retrievability even in case of app restarts or failures.
A_NF_R5	Availability	The system should support offline functionality, allowing farmers to store and access disease reports without internet dependency. The backend should be scalable to handle multiple service requests efficiently, ensuring smooth operation even under high user loads. Future updates and AI model improvements should be deployed seamlessly without disrupting existing functionality.

2.5 SOFTWARE AND HARDWARE REQUIREMENTS

Table 2.3 Software Requirements

Sr No	Requirement Name	Description
1	Operating System	Android 8.0+ required.
2	Frontend Framework	Flutter (Dart) for UI.
3	Backend Framework	FastAPI (Python).
4	Database	MongoDB.
5	AI Model	PyTorch, ONNX Runtime.
6	Security	JWT authentication
7	Offline Optimization	Quantized ONNX-optimized AI models.

Table 2.4 Hardware Requirements

Sr No	Requirement Name	Description
1	Mobile Device	Android 8.0+ support.

2	Camera Quality	Minimum 8MP camera.
3	Processing Power	ARMv7 (32-bit) or ARMv8 (64-bit) processor required.
4	Storage Capacity	At least 500MB free space.
5	Battery Optimization	Optimized AI inference for low power consumption.

3. SYSTEM DESIGN

3.1 BLOCK DIAGRAM

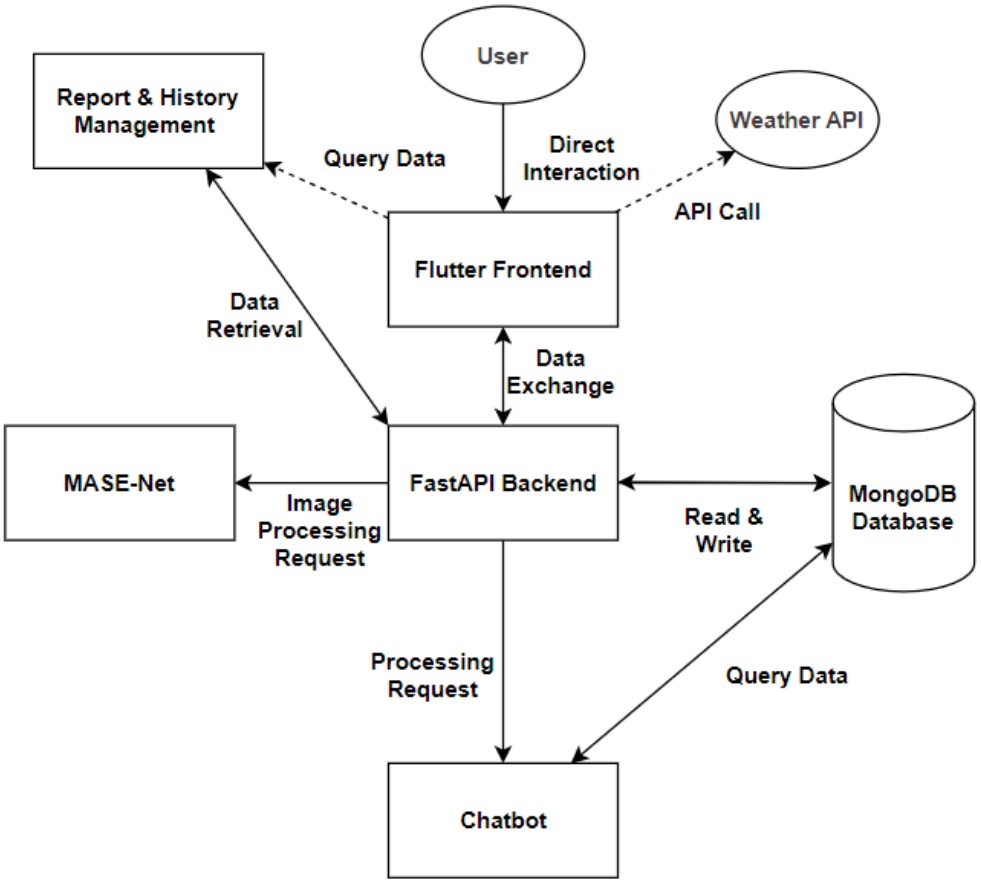


Fig 3.1 Block Diagram

The above diagram depicts the overall system design and its components. The User interacts with the Flutter Frontend, which serves as the interface for disease detection, weather insights, and historical report retrieval. The frontend communicates with the FastAPI Backend, which manages AI inference, database interactions, and chatbot processing. The Weather API provides real-time weather data, helping farmers make informed decisions about environmental conditions. The MASE-Net model processes images sent by the backend, performing disease classification and severity estimation based on affected leaf area. The backend interacts with MongoDB for storing disease reports, user data, and chatbot interactions. The AI-powered chatbot retrieves relevant disease information and treatment suggestions from this database. Additionally, the Report & History Management module allows users to access past disease assessments for reference.

3.2 DATABASE DESIGN

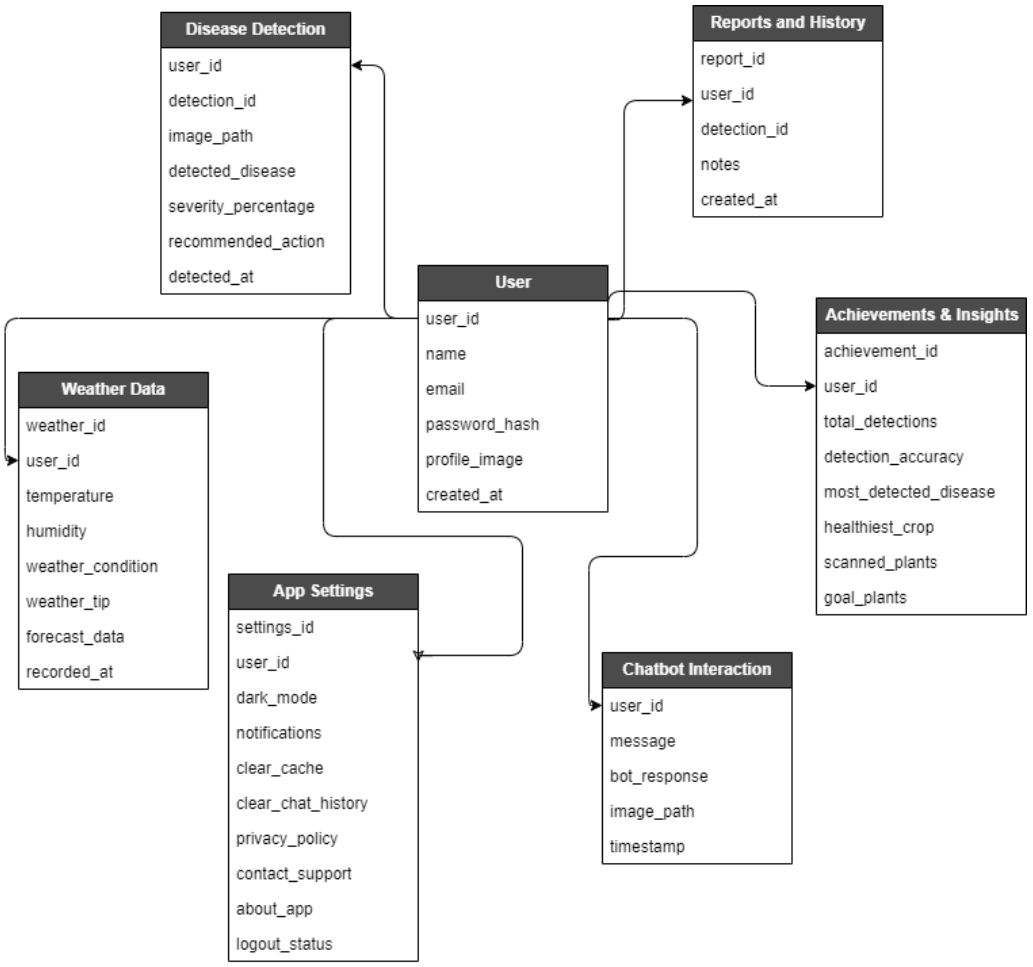


Fig 3.2 Database Design

The above diagram illustrates the table relationship structure of AgriVision, showcasing how different components interact within the system. At the core, the User table serves as the central entity, linking various functionalities such as Disease Detection, Chatbot Interaction, Reports & History, Insights Tracking, and App Settings. The Disease Detection module processes images using MASE-Net for disease classification and severity estimation, storing results for future reference. The Chatbot Interaction table logs user queries and AI-generated responses, ensuring personalized assistance. The Weather API provides real-time weather insights to help farmers make informed decisions, though weather data is not stored in the database. The Reports & History table maintains past disease assessments, allowing users to track previous diagnoses and recommendations, while the Insights Tracking module analyzes historical disease patterns and trends, helping farmers understand recurring issues and optimize farming strategies. Lastly, App Settings

enable users to customize preferences, ensuring a personalized experience. This structured database design ensures efficient data retrieval, seamless interactions, and an organized user experience within the AgriVision ecosystem.

3.3 ER DIAGRAM

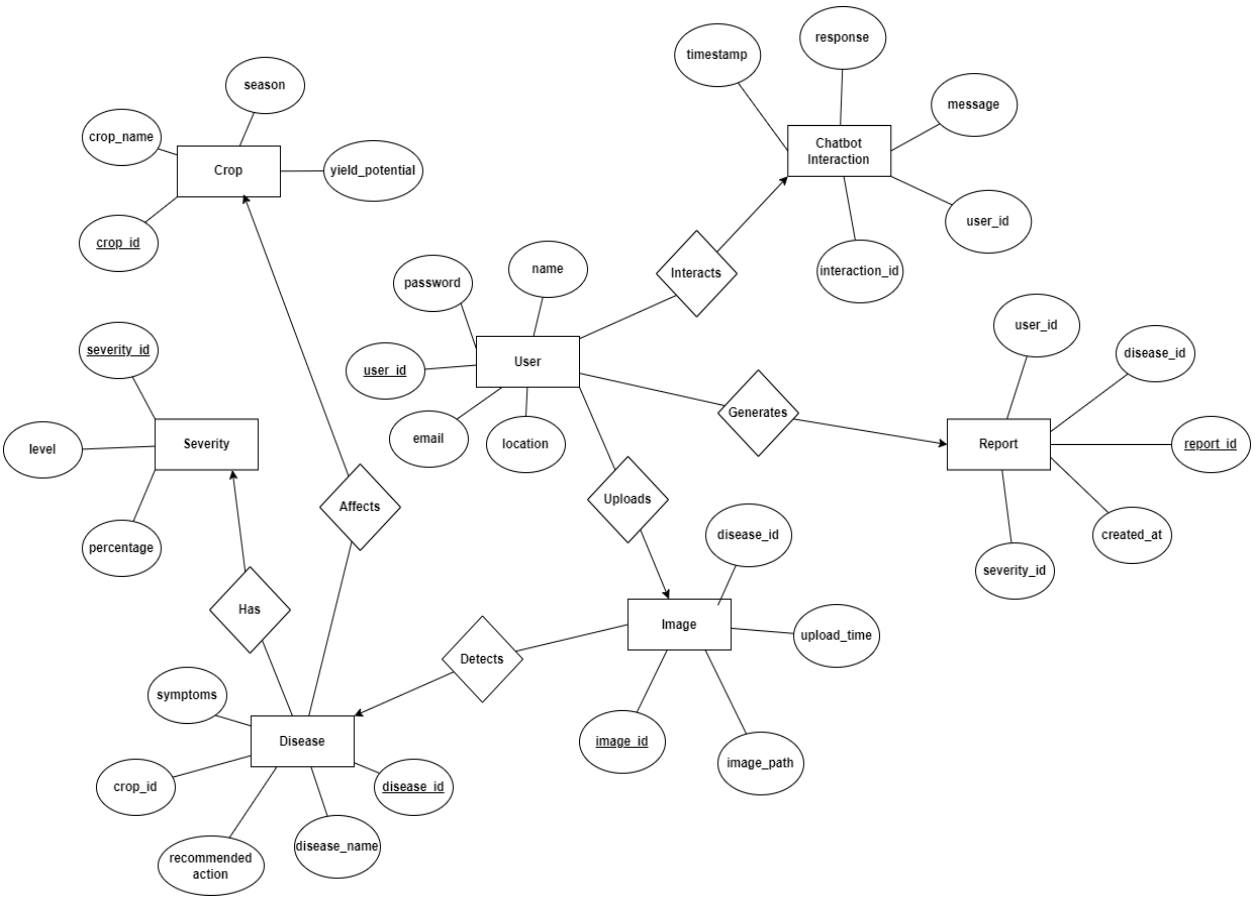


Fig 3.3 Entity Relationship Diagram

The above ER diagram illustrates the key entities in AgriVision and their relationships. Association links define how these entities interact, cardinality represents the number of instances each entity can have, and participation constraints highlight the significance of each entity in the system. Users can upload images, which are processed using the MASE-Net model for disease classification and severity estimation, identifying diseases affecting specific crops. Each disease is associated with symptoms and recommended actions and is classified based on its severity level (Low, Moderate, High) to indicate its impact. Users can generate reports that store disease classification, severity level, and recommended actions for future reference. Additionally, users can interact with the chatbot, which provides real-time insights, farming recommendations, and disease prevention

guidance using FAISS-based retrieval with Sentence-BERT and TinyBERT. The structured relationships between these entities ensure an efficient, intelligent system for crop disease detection, severity assessment, and user assistance, ultimately supporting precision agriculture.

3.4 DATA FLOW DIAGRAM

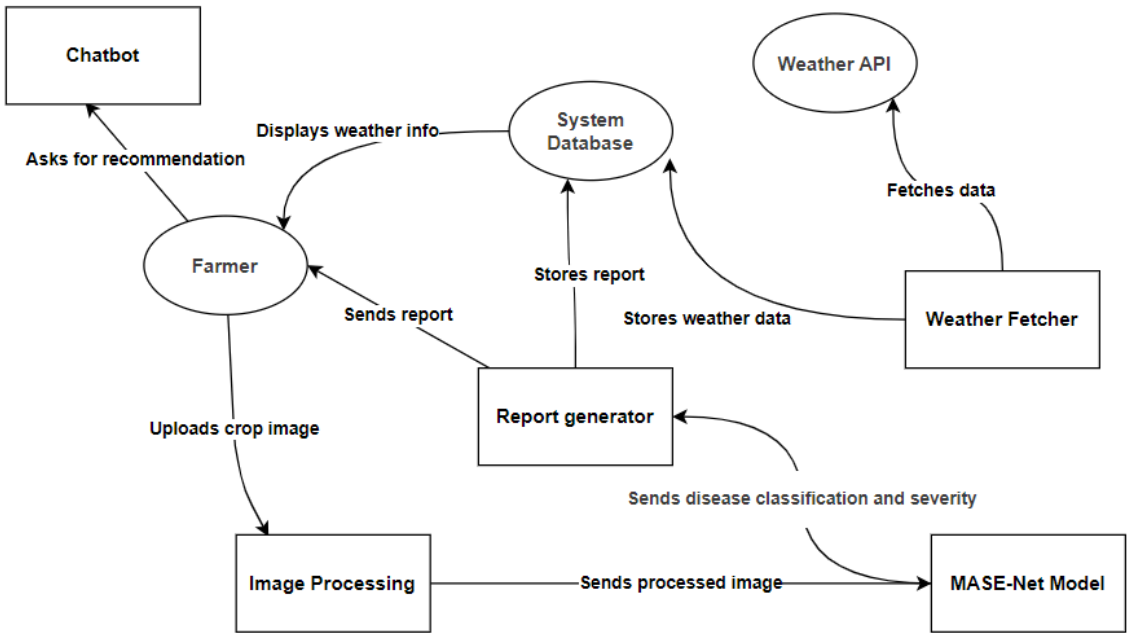


Fig 3.4 Data Flow Diagram

The above figure illustrates the data flow within AgriVision, ensuring accurate crop disease detection, severity estimation, and user assistance. Farmers upload crop images, which are processed by the Image Processing Module and analyzed by MASE-Net to classify diseases and estimate severity (Low, Moderate, or High). The Report Generator compiles these results into structured reports stored in the System Database for tracking and prevention. Farmers can also interact with the AI-powered chatbot, which provides real-time insights, disease prevention tips, and treatment recommendations using FAISS-based retrieval with Sentence-BERT and TinyBERT. Additionally, the Weather Fetcher retrieves real-time weather data from the Weather API to assist in farm management, though this data is not stored. By integrating disease detection, severity estimation, chatbot assistance, and weather insights, AgriVision delivers a comprehensive and intelligent support system for precision agriculture.

3.5 USE CASE DIAGRAM

AgriVision is designed to provide crop health monitoring and agricultural insights to all users, offering equal access to its AI-driven functionalities. Users can upload crop images for disease detection, view detailed reports, interact with the AI-powered chatbot for farming advice, and check real-time weather updates. Whether used by farmers, researchers, traders, or agricultural enthusiasts, AgriVision delivers a seamless and informative experience, ensuring data-driven decision-making in agriculture.

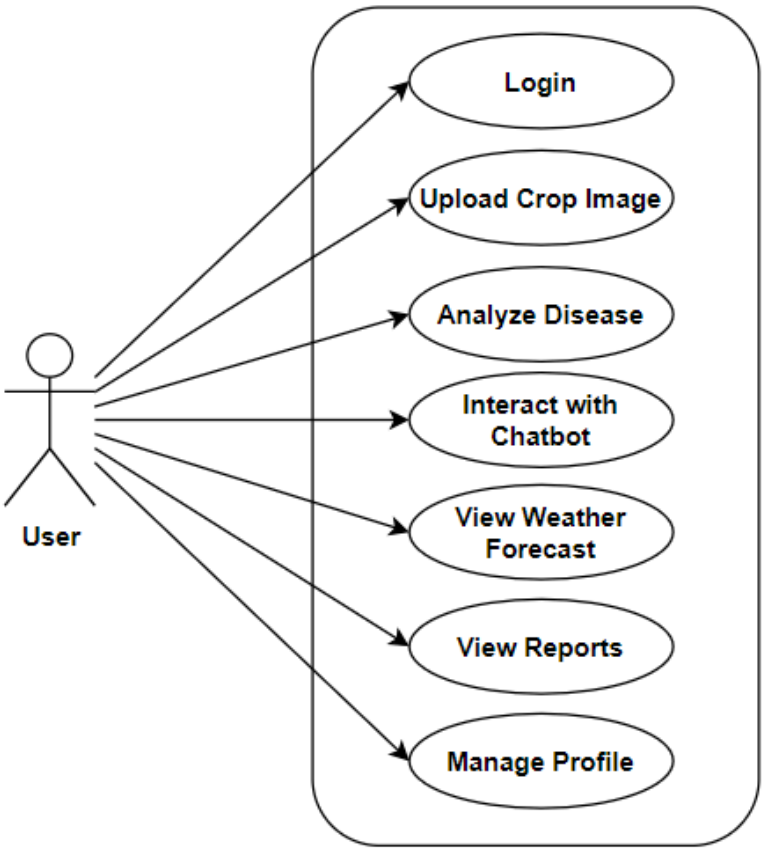


Fig 3.5 Use Case Diagram

The above Use Case Diagram illustrates the interactions between the User and the AgriVision system, highlighting the key functionalities available within the application. Users can log in, upload crop images, analyze diseases, interact with the chatbot, view weather forecasts, access reports, and manage their profile. Each of these functionalities is represented as a distinct use case within the system boundary. The above Use Case Diagram illustrates the interactions between the User and the AgriVision system, highlighting the key functionalities available within the application. Users can log in, upload crop images, analyze diseases, interact with the chatbot, view weather forecasts, access reports, and

manage their profile. Each of these functionalities is represented as a distinct use case within the system boundary.

3.6 USER INTERFACE DIAGRAM

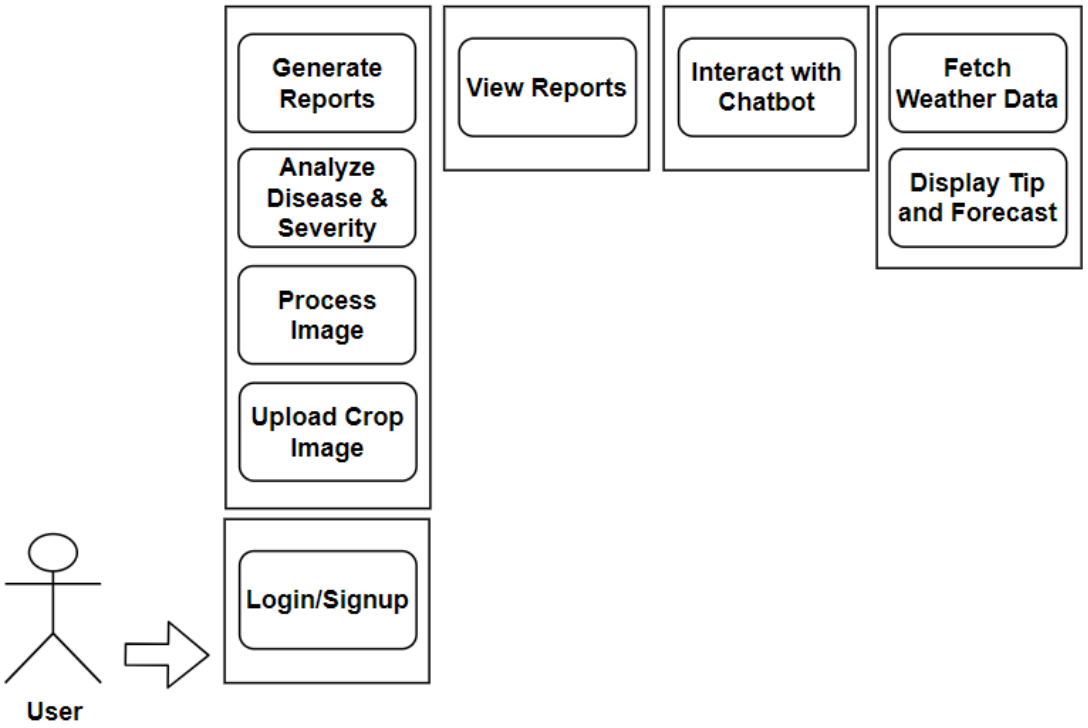


Fig 3.6 User Interface Pictorial Diagram

The above figure illustrates the User Interface Flow of AgriVision, showcasing how users interact with the system's core functionalities. Users first access the Login/Signup module, which grants them entry into the application. They can then upload crop images, which are processed by the MASE-Net Model to classify diseases and estimate severity (Low, Moderate, High). Once analyzed, a structured disease report is generated and stored, allowing users to view past results through the Reports module. Additionally, users can interact with the AI-powered chatbot, which provides farming advice and disease prevention recommendations using FAISS-based retrieval with Sentence-BERT and TinyBERT. The system also fetches real-time weather data from the Weather API and displays it to assist in farm management, ensuring users have relevant environmental insights when making agricultural decisions. AgriVision integrates disease detection, chatbot assistance, and weather insights into a seamless experience, providing an intelligent and accessible tool for precision agriculture.

4. IMPLEMENTATION

4.1 SOURCE CODE

4.1.1 Frontend

The AgriVision frontend is developed using Flutter, a cross-platform framework that provides a seamless and responsive user experience. The app follows a clean and structured UI design, ensuring accessibility and ease of use for all users. Key screens include Login, Signup, Home, Disease Detection, Chatbot, Weather, Reports, Profile, and Settings, each designed with intuitive navigation and real-time data updates. The UI is built using Flutter widgets, with Shared Preferences storing user settings locally, while disease reports and chatbot interactions are managed through the database. The app integrates with the FastAPI backend to retrieve authentication, AI-powered disease detection, chatbot responses, and real-time weather insights, ensuring a smooth and efficient workflow.

4.1.1.1 Login Screen

The Login Screen allows users to authenticate using their email/phone and password. It validates input, sends a login request to the FastAPI backend, and stores the JWT token for future API calls. The screen also provides a forgot password option and navigates users to the Signup or Home screen upon successful login.

```
void _login() async {
  if (_formKey.currentState!.validate()) {
    setState(() => _isLoading = true);
    final response = await http.post(
      Uri.parse('http://localhost:8000/api/login'),
      headers: {"Content-Type": "application/json"},
                          body: json.encode({"email": _emailController.text, "password": _passwordController.text}),
    );
    if (response.statusCode == 200) {
      SharedPreferences prefs = await SharedPreferences.getInstance();
      await prefs.setString('access_token', json.decode(response.body)['access_token']);
            Navigator.pushReplacement(context, MaterialPageRoute(builder: (context) => HomeScreen()));
```

```
    } else {
      Fluttertoast.showToast(msg: "Invalid credentials");
    }
    setState(() => _isLoading = false);
  }
}
```

4.1.1.2 Signup Screen

The Signup Screen allows users to create an account by entering their full name, email, phone number, and password. It validates inputs, sends a signup request to the FastAPI backend, and stores the login status using SharedPreferences. Upon successful registration, users are redirected to the Home Screen.

```
void _signup() async {
  if (_formKey.currentState!.validate()) {
    setState(() => _isLoading = true);

    try {
      final response = await ApiService().signup(
        _fullNameController.text,
        _emailController.text,
        _phoneController.text,
        _passwordController.text,
      );

      Fluttertoast.showToast(msg: "Signup successful!");
      SharedPreferences prefs = await SharedPreferences.getInstance();
      prefs.setBool("isLoggedIn", true);
      Navigator.pushReplacement(context, MaterialPageRoute(builder: (context) =>
HomeScreen()));
    } catch (e) {
      Fluttertoast.showToast(msg: "Signup failed. Please try again.");
    } finally {
      setState(() => _isLoading = false);
```

```
    }
  }
}
```

4.1.1.3 Home Screen

The Home Screen serves as the central hub, providing navigation to key features like disease detection, chatbot, weather updates, and reports. It features an auto-scrolling banner displaying farming tips/alerts and an interactive grid of feature cards. The bottom navigation bar allows users to access settings and profile screens.

```
Widget _buildCard(int index) {
  return InkWell(
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(builder: (context) => cardData[index]["screen"]),
      );
    },
    child: Container(
      width: MediaQuery.of(context).size.width * 0.43,
      height: 120,
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(12),
        image: DecorationImage(
          image: AssetImage(cardData[index]["image"]),
          fit: BoxFit.cover,
        ),
      ),
      child: Stack(
        children: [
          Container(
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(12),
              color: Colors.black.withOpacity(0.5),
```

```
          ),
        ),
        Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(cardData[index]["icon"], size: 40, color: Colors.white),
              SizedBox(height: 8),
              Text(
                cardData[index]["title"],
                style: TextStyle(fontSize: 14, color: Colors.white),
                textAlign: TextAlign.center,
              ),
            ],
          ),
        ),
      ],
    ),
  ),
 );
}
```

4.1.1.4 Settings Screen

The Settings Screen allows users to manage app preferences, including notifications, password changes, clearing cache/chat history, privacy policy, and logout. It uses SharedPreferences to store settings and provides confirmation dialogs for critical actions like logout and data deletion.

```
ListView(
 shrinkWrap: true,
 children: [
  _buildSwitchTile("Notifications", notifications, (value) {
   setState(() {
    notifications = value;
```

```
      });
      _saveSettings();
    }),
    Divider(),
    _buildSettingsOption("Change Password", _changePasswordDialog),
    Divider(),
    _buildSettingsOption("Clear Chat History", _confirmClearChatHistory),
    Divider(),
    _buildSettingsOption("Clear App Cache", _clearAppCache),
    Divider(),
    _buildSettingsOption("Privacy Policy", _showPrivacyPolicy),
    Divider(),
    _buildSettingsOption("Contact Support", _showContactSupport),
    Divider(),
    _buildSettingsOption("About AgriVision", _showAboutAgriVision),
    Divider(),
    _buildSettingsOption("Logout", _confirmLogout, isLogout: true),
  ],
);
```

4.1.1.5 Profile Screen

The Profile Screen displays user details and personalized insights from disease detections. Users can edit their profile, view total detections, accuracy, most detected disease, and healthiest crop, and track progress toward scanned plant goals. Profile data is stored locally using SharedPreferences and insights are fetched from the backend API.

```
Future<void> _editProfile() async {
  TextEditingController nameController = TextEditingController(text: _userName);
  TextEditingController emailController = TextEditingController(text: _userEmail);

  await showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
```

```
      title: Text("Edit Profile"),
      content: Column(
       mainAxisSize: MainAxisSize.min,
       children: [
        TextField(controller: nameController, decoration: InputDecoration(labelText:
"Name")),
        TextField(controller: emailController, decoration: InputDecoration(labelText:
"Email")),
       ],
      ),
      actions: [
       TextButton(onPressed: () => Navigator.pop(context), child: Text("Cancel")),
       ElevatedButton(
        onPressed: () async {
         SharedPreferences prefs = await SharedPreferences.getInstance();
         await prefs.setString('userName', nameController.text);
         await prefs.setString('userEmail', emailController.text);
         setState(() {
          _userName = nameController.text;
          _userEmail = emailController.text;
         });
         Navigator.pop(context);
        },
        child: Text("Save"),
       ),
      ],
     );
    },
   );
}
```

4.1.1.6 Disease Detection Screen

The Disease Detection Screen allows users to upload or capture an image of a plant, which is analyzed using the MASE-Net model to identify diseases and estimate severity. The

results include the detected disease, severity percentage, and recommended actions. The user ID is fetched from SharedPreferences, and API requests are handled via FastAPI.

```
Future<void> _submitImage() async {
  if (_selectedImage == null || userId == null) return;

  setState(() => _loading = true);

  try {
    List<int> imageBytes = await _selectedImage!.readAsBytes();
    String base64Image = base64Encode(imageBytes);
    int parsedUserId = int.tryParse(userId!) ?? 0;

    Map<String, dynamic> response = await predictDisease(parsedUserId, base64Image);

    setState(() {
      _detectedDisease = response["disease"];
      _severity = "${response["severity"].toStringAsFixed(2)}%";
      _recommendedAction = "Follow recommended treatment for $_detectedDisease.";
    });
  } catch (e) {
    setState(() {
      _detectedDisease = "Error: Unable to analyze image";
      _severity = "";
      _recommendedAction = "";
    });
  } finally {
    setState(() => _loading = false);
  }
}
```

4.1.1.7 Chatbot Screen

The Chatbot Screen allows users to interact with Berry, the AgriVision assistant, for agricultural guidance. Users can send text queries or upload an image to get disease

predictions using the MASE-Net model. Chat history is stored in SharedPreferences, and responses are fetched from the FastAPI backend.

```
void _sendMessage() async {
  String userMessage = _messageController.text.trim();
  if (userMessage.isEmpty && _selectedImage == null) return;

  setState(() {
    _messages.add({'text': userMessage, 'image': _selectedImage?.path, 'isUser': true});
    _messageController.clear();
    _selectedImage = null;
  });

  _saveChatHistory();

  try {
    String response = _selectedImage != null
        ? await _getDiseasePrediction(_selectedImage!)
        : await getChatResponse(userId, userMessage);

    setState(() {
      _messages.add({'text': response, 'image': null, 'isUser': false});
    });
    _saveChatHistory();
  } catch (e) {
    setState(() {
      _messages.add({'text': "Error: Unable to fetch response", 'image': null, 'isUser': false});
    });
  }
}
```

4.1.1.8 Weather Screen

The Weather Screen provides real-time weather updates and a 5-day forecast using the OpenWeather API. It retrieves the user's location via Geolocator and displays temperature,

humidity, wind speed, and weather conditions. Additionally, it offers farming tips based on weather conditions.

```
Future<void> _fetchWeather() async {
  setState(() => _loading = true);

  try {
    Position position = await _getLocation();
    double lat = position.latitude, lon = position.longitude;

    final response = await http.get(Uri.parse(

"https://api.openweathermap.org/data/2.5/weather?lat=$lat&lon=$lon&units=metric&appid
=$_apiKey"));

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      setState(() {
        _temperature = "${data['main']['temp']}°C";
        _condition = data['weather'][0]['description'];
        _humidity = "${data['main']['humidity']}%";
        _windSpeed = "${data['wind']['speed']} km/h";
        _icon = _getWeatherIcon(data['weather'][0]['main']);
        _weatherTip = _generateWeatherTip(data['main']['temp'], data['main']['humidity']);
      });
      _fetchForecast(lat, lon);
    } else {
      setState(() => _condition = "Failed to fetch weather");
    }
  } catch (e) {
    setState(() => _condition = "Error: Unable to get weather data.");
  }
}
```

4.1.1.9 Reports Screen

The Reports Screen displays past disease detection reports and chatbot interactions fetched from the backend. Users can view the detected diseases, severity levels, and timestamps of their interactions. Reports are retrieved using the FastAPI backend and stored in the AgriVision database.

```
Future<void> _loadReports() async {
  try {
    List<Map<String, dynamic>> diseaseReports = await fetchDiseaseReports(userId);
    List<Map<String, dynamic>> chatbotLogs = await fetchChatbotLogs(userId);

    setState(() {
      _diseaseReports = diseaseReports;
      _chatbotLogs = chatbotLogs;
      _loading = false;
    });
  } catch (e) {
    setState(() => _loading = false);
  }
}
```

4.1.2 Backend

The AgriVision backend is built using FastAPI, a high-performance web framework for Python, handling user authentication, disease detection, chatbot interactions, real-time weather insights retrieval, and report management. It processes crop disease classification and severity estimation using MASE-Net, leveraging ONNX Runtime for efficient AI inference. The backend stores disease reports, chatbot interactions, and user data in MongoDB, while JWT authentication ensures secure access. It communicates with the Flutter frontend via RESTful APIs, delivering real-time responses for disease analysis, chatbot queries, and weather insights to provide an efficient and responsive agricultural decision-support system.

4.1.2.1 app.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```

```
from auth import auth_router
from chatbot import chat_router
from user_insights import user_router
from reports import reports_router
from detection import disease_router

app = FastAPI(title="AgriVision API")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router)
app.include_router(chat_router)
app.include_router(user_router)
app.include_router(reports_router)
app.include_router(disease_router)
```

4.1.2.2 auth.py

```
import logging
from datetime import datetime, timedelta
from fastapi import APIRouter, HTTPException, Depends
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError  # type: ignore
from passlib.context import CryptContext  # type: ignore
from models import User, UserLogin
from database import users_collection, user_insights_collection

# Initialize router
auth_router = APIRouter()
```

```
# JWT settings
JWT_SECRET = "secret"  # Replace with a secure environment variable
JWT_ALGORITHM = "HS256"
TOKEN_EXPIRE_MINUTES = 10080  # 7 days

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# OAuth2 authentication scheme
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/login")

# Logger setup
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Utility functions
def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict, expires_delta: timedelta = None) -> str:
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=30))
    to_encode = {**data, "exp": expire}
    return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)) -> str:
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        user_id = payload.get("user_id")
        if not user_id:
            raise HTTPException(status_code=401, detail="Invalid authentication token")
        return user_id
```

```
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token or expired session")

# Authentication functions
async def authenticate_user(email: str, password: str):
    user = await users_collection.find_one({"email": email})
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    if not verify_password(password, user["password"]):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({"user_id": str(user["_id"])},
timedelta(minutes=TOKEN_EXPIRE_MINUTES))
    return token, {**user, "_id": str(user["_id"])}  # Ensure _id is a string

async def change_password(user_email: str, old_password: str, new_password: str):
    user = await users_collection.find_one({"email": user_email})
    if not user or not verify_password(old_password, user["password"]):
        raise HTTPException(status_code=400, detail="Incorrect old password")

    if old_password == new_password:
        raise HTTPException(status_code=400, detail="New password must be different from
the old one")

    hashed_new_password = hash_password(new_password)
    await users_collection.update_one({"email": user_email}, {"$set": {"password":
hashed_new_password}})

    return {"message": "Password updated successfully"}

# API Endpoints
@auth_router.post("/api/signup")
async def signup(user: User):
```

```
    if await users_collection.find_one({"email": user.email}):
        return {"message": "Account already exists. Please login."}

    hashed_password = hash_password(user.password)
    new_user = {
        "username": user.username,
        "email": user.email,
        "phone": user.phone,
        "password": hashed_password,
        "created_at": datetime.utcnow(),
    }

    result = await users_collection.insert_one(new_user)
    user_id = str(result.inserted_id)

    # Initialize user insights
    await user_insights_collection.insert_one({
        "_id": user_id,
        "totalDetections": 0,
        "detectionAccuracy": 0.0,
        "mostDetectedDisease": "None",
        "healthiestCrop": "Unknown",
        "scannedPlants": 0,
        "goalPlants": 0,
    })

    token = create_access_token({"user_id": user_id},
timedelta(minutes=TOKEN_EXPIRE_MINUTES))
    return {"message": "User created successfully!", "user_id": user_id, "token": token}

@auth_router.post("/api/login")
async def login(login: UserLogin):
    user = await users_collection.find_one({"email": login.email})
    if not user or not verify_password(login.password, user["password"]):
```

```
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({"user_id": str(user["_id"])},
timedelta(minutes=TOKEN_EXPIRE_MINUTES))
    return {"message": "Login successful!", "user_id": str(user["_id"]), "token": token}

@auth_router.post("/api/forgot_password")
async def forgot_password(email: str):
    user = await users_collection.find_one({"email": email})
    if not user:
        raise HTTPException(status_code=404, detail="Email not found")

    reset_link = f"http://192.168.1.12:8000/reset_password/{email}"
    return {"message": "Password reset link sent to your email.", "reset_link": reset_link}
```

4.1.2.3 chatbot.py

```
from fastapi import APIRouter
from database import chatbot_collection
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

chat_router = APIRouter()

# Load the fine-tuned Sentence-BERT model
bert_model = SentenceTransformer("C:\\Users\\raksh\\agrivision\\fine-tuned-agrivision")

# Normalize embeddings for cosine similarity
def normalize(vectors):
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)
    return vectors / norms

# FAQ Dataset
faq_data = [
```

```
    {"question": "What is late blight?", "answer": "Late blight is a fungal disease caused by Phytophthora infestans, primarily affecting potatoes and tomatoes. It causes dark lesions on leaves and stems."},
    {"question": "How to treat powdery mildew?", "answer": "Use neem oil, sulfur, or copper-based fungicides to treat powdery mildew. Ensure proper air circulation and avoid overhead watering."},
    {"question": "What is early blight?", "answer": "Early blight is a fungal disease that primarily affects tomatoes and potatoes. It causes circular lesions on leaves and can lead to defoliation."},
    {"question": "What are the symptoms of leaf spot?", "answer": "Leaf spot is characterized by small, round, or angular spots with dark borders, often surrounded by yellow halos. It is caused by various fungi and bacteria."},
    {"question": "How do I prevent rust in plants?", "answer": "Rust can be prevented by ensuring good air circulation, removing infected leaves, and applying fungicides. Resistant plant varieties are also available."},
]

# Convert FAQ questions to embeddings
questions = [item["question"] for item in faq_data]
embeddings = bert_model.encode(questions, normalize_embeddings=True)

# Initialize FAISS index
if embeddings.shape[0] > 0:
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
else:
    index = None  # Prevents FAISS crash if no data is available

@chat_router.get("/chatbot")
async def chatbot(query: str):
    """Retrieves the best-matching FAQ response using FAISS similarity search."""

    if index is None or index.ntotal == 0:
        return {"response": "Chatbot database is empty. Please update the dataset."}
```

```
    # Encode and normalize query embedding
    query_embedding = bert_model.encode([query], normalize_embeddings=True)

    # Retrieve top 3 matches
    distances, indices = index.search(query_embedding, k=3)

    # Define similarity threshold
    threshold = 0.7
    best_match_idx, best_score = None, -1

    for i, idx in enumerate(indices[0]):
        if idx != -1 and distances[0][i] >= threshold and distances[0][i] > best_score:
            best_match_idx, best_score = idx, distances[0][i]

    # Handle no valid match scenario
    if best_match_idx is None:
        response = "I'm sorry, I couldn't find a precise answer. Please try rephrasing."
    else:
        response = faq_data[best_match_idx]["answer"]

    # Debugging Logs
    print(f"\nUser Query: {query}")
    for i, idx in enumerate(indices[0]):
        if idx != -1:
            print(f'Candidate {i+1}: {faq_data[idx]['question']} (Score: {distances[0][i]:.2f})")

    print(f"Matched Question: {faq_data[best_match_idx]['question']} (Score:
{best_score:.2f})")

    # Store in Database
    chatbot_collection.insert_one({"query": query, "response": response})

    return {"response": response}
```

4.1.2.4 database.py

```
from motor.motor_asyncio import AsyncIOMotorClient # type: ignore

mongo_client = AsyncIOMotorClient("mongodb://localhost:27017")
mongo_db = mongo_client["agrivision"]

users_collection = mongo_db["users"]
user_insights_collection = mongo_db["user_insights"]
chatbot_collection = mongo_db["chatbot_logs"]
reports_collection = mongo_db["disease_reports"]
```

4.1.2.5 detection.py

```
from fastapi import APIRouter, File, UploadFile
from database import reports_collection
import onnxruntime as ort
import numpy as np
from PIL import Image
from datetime import datetime
import io

disease_router = APIRouter()

# Load ONNX Model
ort_session = ort.InferenceSession("masenet.onnx")

# Preprocess Image for MASE-Net
def preprocess_image(image_file):
    image = Image.open(image_file).convert("RGB")  # Ensure RGB format
    image = image.resize((224, 224))
    image = np.array(image, dtype=np.float32) / 255.0  # Normalize to [0,1]
    image = np.transpose(image, (2, 0, 1))  # Change format to (C, H, W)
    return np.expand_dims(image, axis=0)  # Add batch dimension

# Disease label mapping
```

```
disease_labels = {
    0: "Bacterial Leaf Blight", 1: "Bacterial Spot", 2: "Brown Spot",
    3: "Cercospora Leaf Spot", 4: "Common Rust", 5: "Early Blight",
    6: "Healthy", 7: "Late Blight", 8: "Leaf Blast", 9: "Leaf Mold",
    10: "Leaf Scald", 11: "Mosaic Virus", 12: "Northern Leaf Blight",
    13: "Scorch", 14: "Septoria", 15: "Septoria Leaf Spot",
    16: "Sheath Blight", 17: "Spider Mites", 18: "Stripe Rust",
    19: "Target Spot", 20: "Yellow Leaf Curl Virus"
}

# Categorize severity
def categorize_severity(severity):
    return "Low" if severity < 0.3 else "Moderate" if severity < 0.7 else "High"

@disease_router.post("/predict")
async def predict_disease(image: UploadFile = File(...)):
    # Read image file
    image_bytes = await image.read()
    image_file = io.BytesIO(image_bytes)

    # Preprocess the image
    image = preprocess_image(image_file)

    # Perform inference
    input_name = ort_session.get_inputs()[0].name
    result = ort_session.run(None, {input_name: image})

    # Extract predictions
    disease_label = int(result[0].argmax())  # Disease classification
    severity = float(result[1][0])  # Severity estimation
    severity_category = categorize_severity(severity)

    # Store result in MongoDB
    report = {
```

```
        "disease": disease_labels.get(disease_label, "Unknown"),
        "severity": severity_category,
        "severity_value": severity,
        "timestamp": datetime.utcnow()
    }
    reports_collection.insert_one(report)

    return {"disease": disease_labels.get(disease_label, "Unknown"), "severity":
severity_category}
```

4.1.2.6 models.py

```
from pydantic import BaseModel
from datetime import datetime
from pydantic import BaseModel

class User(BaseModel):
    username: str
    email: str
    phone: str
    password: str

    class Config:
        from_attributes = True  # Fix for Pydantic v2

class UserLogin(BaseModel):
    email: str
    password: str

# Pydantic model for validation and serialization (API model)
class Report(BaseModel):
    user_id: str
    disease: str
    severity: float
    date: datetime
```

```
    class Config:
        orm_mode = True  # This allows Pydantic to work with MongoDB objects (e.g., 
ObjectId)

# ChatbotLog model to be returned from API
class ChatbotLog(BaseModel):
    user_id: str
    message: str
    timestamp: str  # ISO formatted date string

    class Config:
        orm_mode = True

# MongoDB-compatible class for storing reports in the database
class ReportDB:
    def __init__(self, user_id: str, crop: str, disease: str, severity: float, date: datetime):
        self.user_id = user_id
        self.crop = crop
        self.disease = disease
        self.severity = severity
        self.date = date

    # Method to convert MongoDB object to Pydantic model (Report)
    def to_pydantic(self):
        return Report(
            user_id=self.user_id,
            crop=self.crop,
            disease=self.disease,
            severity=self.severity,
            date=self.date
        )

    # Method to convert Pydantic model to MongoDB document (ReportDB)
```

```
    @classmethod
    def from_pydantic(cls, report: Report):
        return cls(
            user_id=report.user_id,
            crop=report.crop,
            disease=report.disease,
            severity=report.severity,
            date=report.date
        )

    # Convert MongoDB ObjectId to string (for serializing)
    def to_dict(self):
        report_dict = self.__dict__
        report_dict["_id"] = str(report_dict.get("_id", None))  # Convert ObjectId to string
        return report_dict
```

4.1.2.7 reports.py

```
from fastapi import APIRouter, HTTPException, Depends
from typing import List
from auth import get_current_user
from database import reports_collection, chatbot_collection
from models import ChatbotLog

reports_router = APIRouter()

@reports_router.get("/reports", response_model=List[dict])
async def get_reports(user_id: str = Depends(get_current_user)):
    try:
        reports_cursor = reports_collection.find({"user_id": user_id})
        reports = await reports_cursor.to_list(length=100)

        if not reports:
            raise HTTPException(status_code=404, detail="No reports found for this user.")
```

```
        # Return only required fields
        return [
            {
                "disease": report["disease"],
                "severity": report["severity"],
                "severity_value": report["severity_value"],
                "date" : report["date"],
            }
            for report in reports
        ]

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error fetching reports: {str(e)}")


@reports_router.get("/chatbot_logs", response_model=List[ChatbotLog])
async def get_chatbot_logs(user_id: str = Depends(get_current_user)):
    try:
        # Fetch chatbot logs asynchronously
        logs_cursor = chatbot_collection.find({"user_id": user_id})
        logs = await logs_cursor.to_list(length=100)  # Limit to 100 logs for performance
        return logs
    except Exception as e:
        raise HTTPException(status_code=500, detail="Error fetching chatbot logs: " + str(e))
```

4.1.2.8 user_insights.py

```
from fastapi import APIRouter, HTTPException, Depends
from bson.objectid import ObjectId # type: ignore
from auth import get_current_user
from database import users_collection, user_insights_collection  # Import user insights
collection

user_router = APIRouter()

@user_router.get("/api/user/insights")
```

```
async def get_user_insights(user_id: str = Depends(get_current_user)):  # Keep user_id as string
    # Fetch user insights by string ID (without converting to ObjectId)
    user_insights = await user_insights_collection.find_one({"_id": user_id})
    users = await users_collection.find_one({"_id": ObjectId(user_id)})

    if not user_insights:
        raise HTTPException(status_code=404, detail="User insights not found")

    return {
        "name": users.get("username"),
        "email": users.get("email"),
        "totalDetections": user_insights.get("totalDetections", 0),
        "detectionAccuracy": user_insights.get("detectionAccuracy", 0.0),
        "mostDetectedDisease": user_insights.get("mostDetectedDisease", "Unknown"),
        "healthiestCrop": user_insights.get("healthiestCrop", "Unknown"),
        "scannedPlants": user_insights.get("scannedPlants", 0),
        "goalPlants": user_insights.get("goalPlants", 0),
    }
```

4.1.3 MASE-Net

4.1.3.1 Model Architecture

```
import torch
import torch.nn as nn
import timm

class MASENet(nn.Module):
    def __init__(self, num_classes):
        super(MASENet, self).__init__()

        # Feature Extractor: MobileNetV3
        self.backbone = timm.create_model("mobilenetv3_large_100", pretrained=True, num_classes=0, global_pool="avg")
        backbone_out_features = 1280
```

```
        # Transformer Block: MobileViT
        self.transformer = timm.create_model("mobilevit_s", pretrained=True, num_classes=0,
global_pool="avg")
        transformer_out_features = 640

        # Combined Feature Size
        combined_feature_size = backbone_out_features + transformer_out_features  # 1280 +
640 = 1920

        # Classification Head
        self.classification_head = nn.Sequential(
            nn.Linear(combined_feature_size, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes),
            nn.Softmax(dim=1)
        )

        # Severity Embedding (for contrastive learning)
        self.severity_embedding = nn.Sequential(
            nn.Linear(combined_feature_size, 128),
            nn.ReLU(),
            nn.Linear(128, 64)  # Embedding for contrastive loss
        )

        # Severity Regression (Huber Loss)
        self.severity_head = nn.Sequential(
            nn.Linear(64, 1),
            nn.Sigmoid()  # Ensures severity is in [0,1], scaled to [0,100] later
        )

    def forward(self, x, return_features=False):
        """ Forward pass with optional feature extraction. """
```

```
        backbone_features = self.backbone(x)
        transformer_features = self.transformer(x)
        combined_features = torch.cat([backbone_features, transformer_features], dim=1)

        if return_features:
            return combined_features  # Returns features for clustering

        class_output = self.classification_head(combined_features)
        severity_embedding = self.severity_embedding(combined_features)
        severity_output = self.severity_head(severity_embedding) * 100  # Scale to [0, 100]

        return class_output, severity_output, severity_embedding  # Return embedding for
contrastive loss

    def extract_features(self, x):
        """ Extract features for clustering or downstream tasks. """
        with torch.no_grad():
            backbone_features = self.backbone(x)
            transformer_features = self.transformer(x)
            return torch.cat([backbone_features, transformer_features], dim=1)  # Returns
feature vector
```

4.1.3.2 Training Loop

```
import torch
import matplotlib.pyplot as plt

def train_model(model, train_loader, val_loader, optimizer, scheduler, criterion_class,
criterion_severity,
                epochs=30, grad_accum_steps=2, patience=3):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    best_val_loss = float("inf")
    no_improve_epochs = 0
```

```
use_amp = torch.cuda.is_available()
scaler = torch.cuda.amp.GradScaler() if use_amp else None

# Track loss and accuracy values for visualization
train_losses, val_losses = [], []
train_accuracies, val_accuracies = [], []

for epoch in range(epochs):
    model.train()
    total_loss, correct, total = 0.0, 0, 0

    optimizer.zero_grad()

    # Training Loop
    for step, (images, labels, severity) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)
        severity = severity.to(device).float().unsqueeze(1) / 100  # Normalize severity [0,1]

        with torch.cuda.amp.autocast() if use_amp else torch.enable_grad():
            class_preds, severity_preds, _ = model(images)

            # Ensure severity_preds has requires_grad=True
            severity_preds = severity_preds.float()

            loss_class = criterion_class(class_preds, labels).mean()
            loss_severity = criterion_severity(severity_preds, severity).mean()
            loss = loss_class + loss_severity

        # Gradient Scaling with AMP
        if use_amp:
            scaler.scale(loss).backward()
        else:
            loss.backward()
```

```
        if (step + 1) % grad_accum_steps == 0:
            if use_amp:
                scaler.step(optimizer)
                scaler.update()
            else:
                optimizer.step()
            optimizer.zero_grad()

        # Track Training Metrics
        total_loss += loss.item() * images.size(0)
        correct += (class_preds.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

    train_loss = total_loss / total
    train_acc = (correct / total) * 100
    train_losses.append(train_loss)
    train_accuracies.append(train_acc)

    # Validation Loop
    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0

    with torch.no_grad():
        for images, labels, severity in val_loader:
            images, labels = images.to(device), labels.to(device)
            severity = severity.to(device).float().unsqueeze(1) / 100

            class_preds, severity_preds, _ = model(images)
            severity_preds = severity_preds.float()

            loss_class = criterion_class(class_preds, labels).mean()
            loss_severity = criterion_severity(severity_preds, severity).mean()
            loss = loss_class + loss_severity
```

```
            val_loss += loss.item() * images.size(0)
            val_correct += (class_preds.argmax(dim=1) == labels).sum().item()
            val_total += labels.size(0)

        val_loss /= val_total
        val_acc = (val_correct / val_total) * 100
        val_losses.append(val_loss)
        val_accuracies.append(val_acc)

        print(f"Epoch [{epoch+1}/{epochs}] | Train Loss: {train_loss:.4f}, Train Acc:
{train_acc:.2f}% | "
              f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

        # Update Learning Rate Scheduler
        scheduler.step(val_loss)

        # Early Stopping Mechanism
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            no_improve_epochs = 0
            torch.save(model.state_dict(), "best_model.pth")
        else:
            no_improve_epochs += 1
            if no_improve_epochs >= patience:
                print("Early stopping triggered.")
                break

    return train_losses, val_losses, train_accuracies, val_accuracies

# Start Training
train_losses, val_losses, train_accuracies, val_accuracies = train_model(
    model, train_loader, val_loader, optimizer, scheduler,
    criterion_class, criterion_severity, epochs=30, patience=3
)
```

4.1.3.2 Evaluation

```
import torch
from sklearn.metrics import classification_report, mean_absolute_error, r2_score

def evaluate_model(model, test_loader, label_map):
    """ Evaluates the model on test data. """
    model.eval()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    y_true, y_pred = [], []
    severity_true, severity_pred = [], []

    with torch.no_grad():
        for images, labels, severity in test_loader:
            images, labels = images.to(device), labels.to(device)
            severity = severity.to(device).float()

            class_preds, severity_preds, _ = model(images)
            predicted_labels = class_preds.argmax(dim=1)

            # Store values
            y_true.extend(labels.cpu().numpy())
            y_pred.extend(predicted_labels.cpu().numpy())
            severity_true.extend(severity.cpu().numpy())
            severity_pred.extend(severity_preds.cpu().numpy().flatten() * 100)  # Convert to
percentage

    # Classification Report
    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, target_names=list(label_map.keys())))

    # Severity Estimation Metrics
    mae = mean_absolute_error(severity_true, severity_pred)
    r2 = r2_score(severity_true, severity_pred) if len(set(severity_true)) > 1 else 0  # Prevent
```

error

```
    print("\nSeverity Estimation Metrics:")
    print(f"MAE (Mean Absolute Error): {mae:.4f}")
    print(f"R² Score: {r2:.4f}")

    # Sample Severity Predictions
    print("\nSample Severity Predictions:")
    for i in range(min(10, len(severity_true))):
        print(f"True: {severity_true[i]:.2f}% | Predicted: {severity_pred[i]:.2f}%")

# Run Evaluation
evaluate_model(model, test_loader, label_map)
```

4.2 SCREENSHOTS

Fig 4.1 Splash Screen

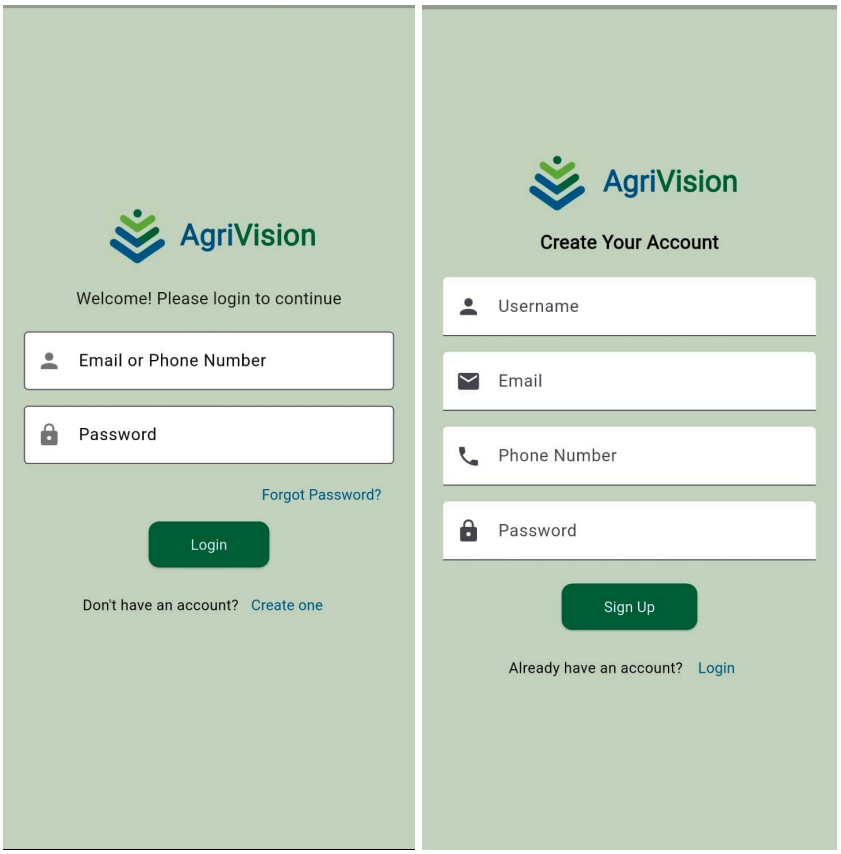


Fig 4.2 Login (left) and Signup (right) Screens

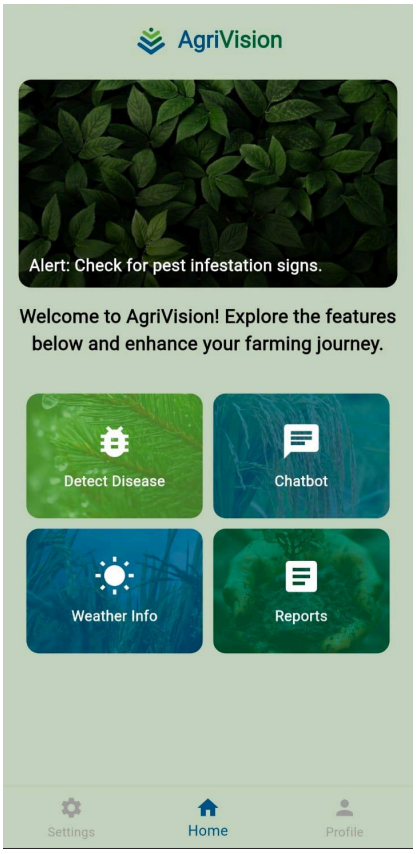


Fig 4.3 Home Screen

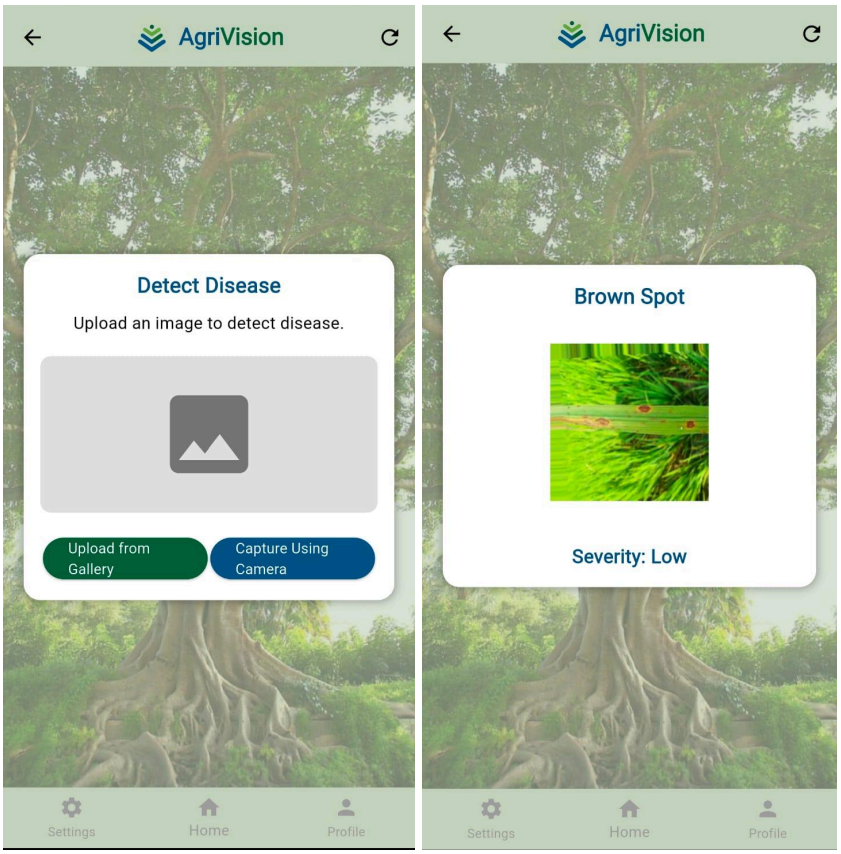


Fig 4.4 Disease Detection Screen

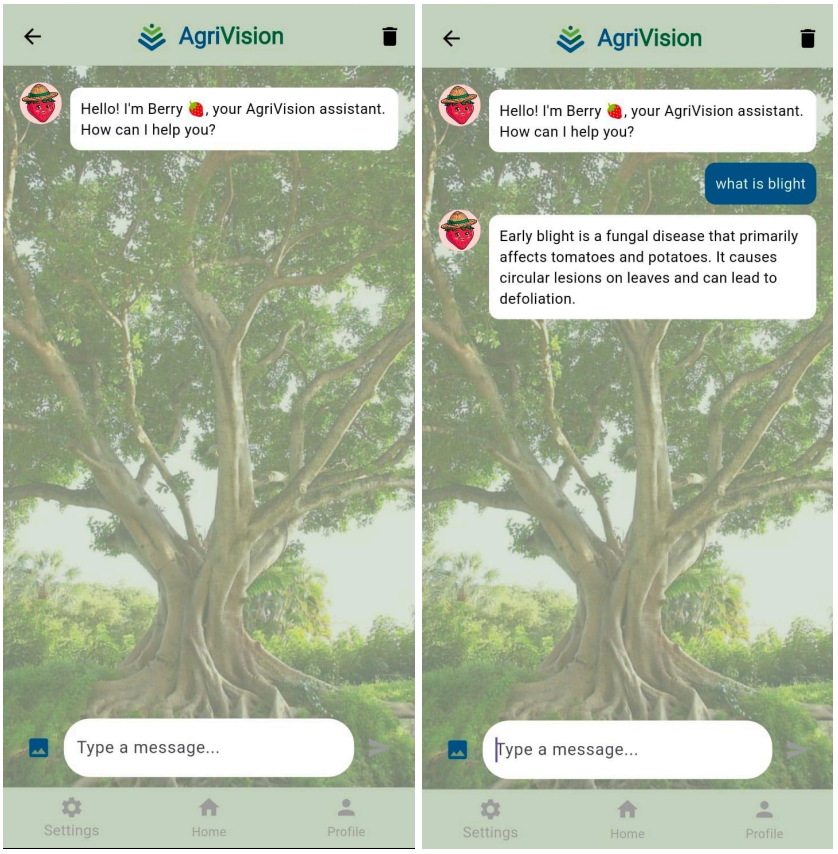


Fig 4.5 Chatbot Screen

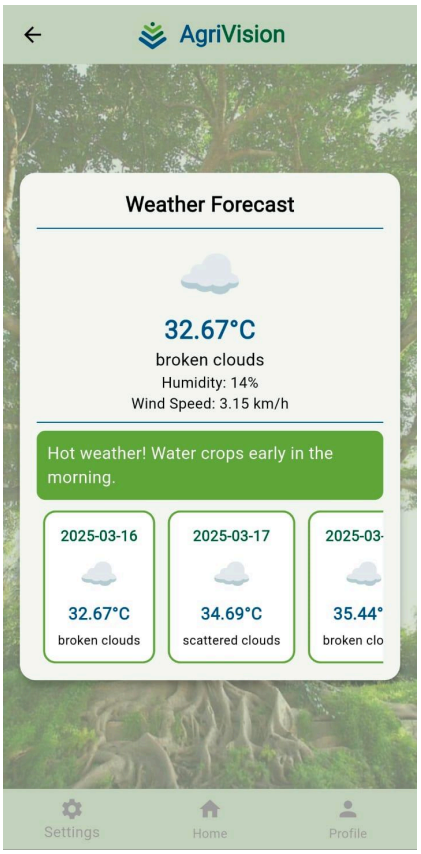


Fig 4.6 Weather Screen

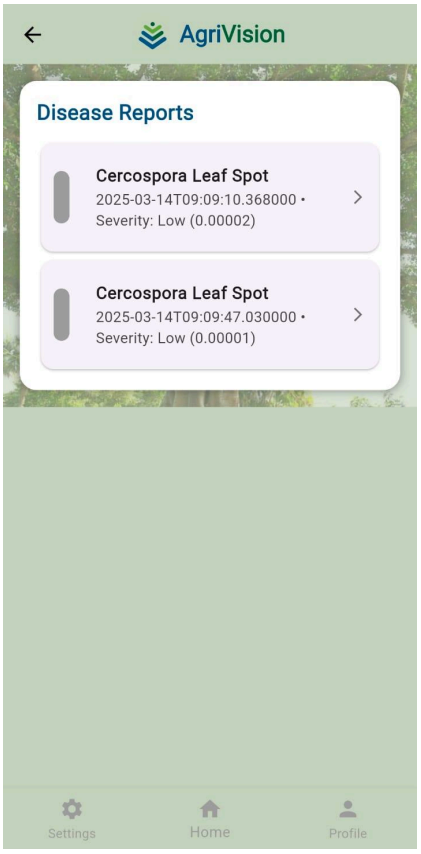


Fig 4.7 Reports Screen

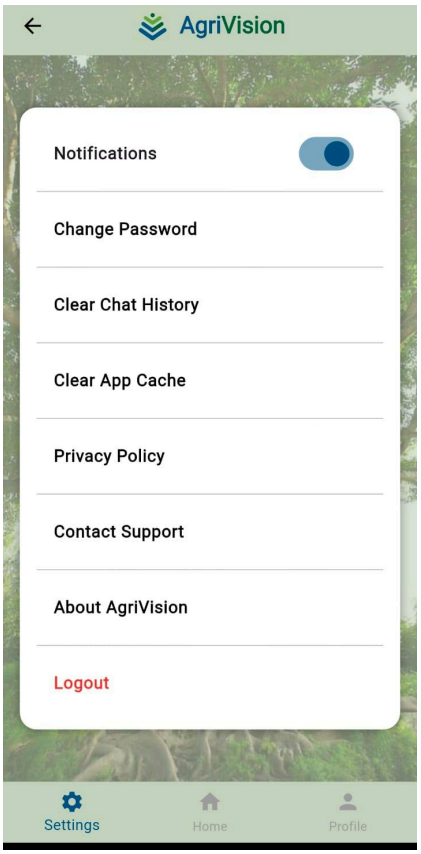


Fig 4.8 Settings Screen

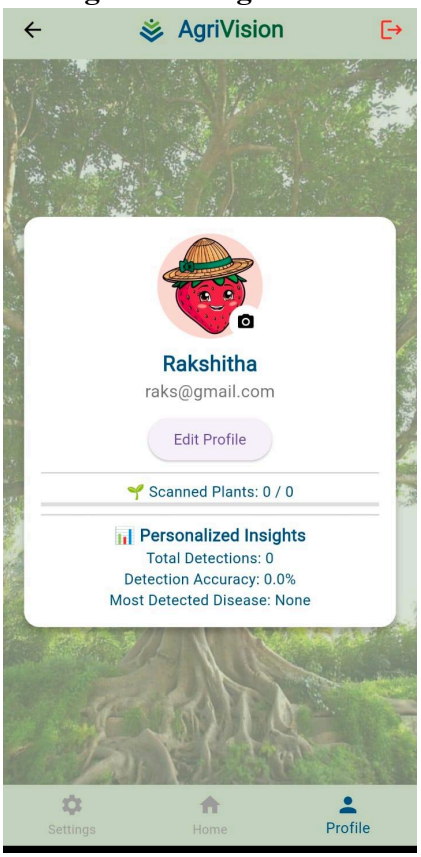


Fig 4.9 Profile Screen

5. TESTING

5.1 TESTING STRATEGIES

Testing plays a critical role in ensuring that a software system meets its functional requirements and performs as expected in real-world environments. For AgriVision, a robust testing strategy was implemented, focusing on unit testing, functional testing, and integration testing. These approaches ensure that individual components work correctly in isolation, the complete system functions as intended, and interactions between various subsystems are seamless. The testing process was thorough, covering the disease detection model, backend system, mobile user interface, and overall system performance.

5.1.1 Unit Testing

Unit testing validates that each individual component functions correctly in isolation, focusing on small, isolated pieces of the application, such as functions, methods, and classes. The objective is to ensure that each component performs as expected and returns the correct results for valid inputs. In AgriVision, unit tests were implemented for core components like the image processing pipeline, disease classification model, and severity estimation algorithms.

For example, unit tests for the image upload function verified that the application correctly processes supported image formats (JPG, PNG) and rejects unsupported formats (TIFF, BMP). The disease classification model was tested with known input images to confirm correct identification and classification of diseases. Similarly, unit tests for the severity estimation algorithm ensured that it correctly categorizes disease severity as Low, Moderate, or High based on the affected leaf area. These tests covered common cases as well as edge cases, such as corrupted images, unsupported formats, and images with no visible disease symptoms.

5.1.2 Functional Testing

Functional testing verifies that the software operates according to its requirements, ensuring that all application components work together correctly. Unlike unit testing, which isolates functions, functional testing evaluates the system as a whole by simulating real-world user workflows, such as image upload, disease detection, severity estimation, and report generation.

For AgriVision, functional tests ensured that the user interface responds correctly to inputs, such as navigating through screens after an image is uploaded or displaying disease diagnosis results accurately. These tests also included simulating mobile user interactions, such as uploading an image, receiving a disease diagnosis, and generating a report with severity estimation. The chatbot functionality was tested to verify that it retrieves relevant responses using FAISS-based retrieval with Sentence-BERT and TinyBERT and provides contextually appropriate agricultural insights.

Functional testing also validated AgriVision's integration with its FastAPI backend, ensuring that requests for disease classification, severity estimation, and chatbot responses were processed accurately. The backend responses were compared against expected outputs to confirm the correctness of disease diagnosis and severity categorization. Additionally, all mobile UI elements, including buttons, input fields, and forms, were tested across various screen sizes and device configurations to ensure a consistent and responsive user experience.

5.1.3 Integration Testing

Integration testing is vital for validating the end-to-end flow of AgriVision, ensuring that different system modules work together seamlessly. This phase focused on ensuring effective communication between the frontend, backend, AI models, and MongoDB database, verifying that data passed between components is correctly processed and displayed.

A key aspect of integration testing was verifying that the Flutter frontend and FastAPI backend functioned smoothly. This included ensuring that the app correctly sends image data to the backend, the backend processes the data using the MASE-Net model for disease classification, and the predictions are returned and displayed correctly on the frontend. Similarly, the integration of the severity estimation module was tested to ensure that it categorizes disease severity correctly as Low, Moderate, or High and that the results are accurately reflected in the app.

Integration testing also validated MongoDB's functionality, ensuring that disease reports, chatbot interactions, and user data were correctly stored and retrieved. This was crucial for allowing users to access past reports through the mobile app. Additional test cases ensured that user login credentials were securely managed using JWT authentication, verifying secure access across multiple sessions. By thoroughly testing these integrations, AgriVision

ensures a reliable and efficient user experience.

5.2 TEST CASES AND REPORTS

5.2.1 Unit Testing

Table 5.1 Unit Testing

Sr No	Module	Test Case	Expected Output	Actual Output	Status
1	Image Upload	Test image upload with supported formats	Image is uploaded successfully	Image uploaded correctly	Pass
2	Disease Classification	Test disease detection on a tomato leaf image	Disease is correctly classified	Correct classification	Pass
3	Severity Estimation	Validate severity estimation for a wheat image	Severity is categorized correctly (Low, Moderate, High)	Correct categorization	Pass
4	Image Upload	Test image upload with unsupported format (TIFF)	Error message displayed	Error message displayed	Pass

5.2.2 Functional Testing

Table 5.2 Functional Testing

Sr No	Module	Test Case	Expected Output	Actual Output	Status
1	Image	Upload a	Image uploaded	Image displayed	Pass

	Upload	valid tomato leaf image	successfully and displayed for classification	correctly	
2	Disease Diagnosis	Upload a maize leaf image for disease classification	Disease identified and classification result displayed	Disease identified correctly	Pass
3	Severity Estimation	Upload a wheat leaf image for severity estimation	Severity level (Low, Moderate, High) displayed correctly	Severity level displayed correctly	Pass
4	Chatbot	Ask about common agricultural practices	Response retrieved from FAISS-based knowledge base with relevant agricultural tips	Correct response provided	Pass

5.2.3 Integration Testing

Table 5.3 Integration Testing

Sr No	Module	Test Case	Expected Output	Actual Output	Status
1	Frontend - Backend	Test disease classification and severity estimation flow	Disease classification and severity level (Low, Moderate, High) displayed after	Correct results displayed	Pass

			image upload		
2	API	Verify API response with disease and severity data	200 OK status and correct disease classification and severity level returned	Correct response with data	Pass
3	Database	Verify report storage in the database	Report is saved correctly in MongoDB	Report saved correctly	Pass

6. CONCLUSION

The AgriVision project has successfully developed an AI-powered crop disease detection and severity estimation system, integrating deep learning with mobile AI to provide an efficient and accessible solution for farmers. By leveraging a CNN-Transformer hybrid architecture in the form of MASE-Net, AgriVision ensures high accuracy in classifying plant diseases and categorizing severity as Low, Moderate, or High based on affected leaf area. The system's mobile-friendly design, achieved through model optimization techniques such as quantization and pruning, enables real-time inference on low-power devices, making it a practical tool for agricultural disease management. The integration of a Flutter-based frontend with a FastAPI backend ensures smooth user interactions, allowing users to upload images, receive disease diagnoses, and access chatbot assistance using FAISS-based retrieval with Sentence-BERT and TinyBERT.

Despite its advancements, challenges such as dataset limitations, model interpretability, and adaptation to diverse environmental conditions remain. The effectiveness of AI-driven disease detection depends on the availability of high-quality, regionally diverse training data, and future improvements should focus on self-supervised learning techniques to enhance model generalization. Additionally, while deep learning models achieve high accuracy, their complexity necessitates explainability techniques to improve transparency and trust among farmers and agricultural stakeholders. Another key area for improvement is expanding AgriVision to support a wider range of crops and disease variants, ensuring that the system remains adaptable to evolving agricultural threats.

In conclusion, AgriVision represents a significant advancement in AI-powered precision agriculture, bringing state-of-the-art deep learning techniques into the hands of farmers through an intuitive and efficient mobile application. By ensuring real-time disease detection and severity estimation, AgriVision empowers users with actionable insights, ultimately contributing to sustainable farming practices and improved crop yields. With continued innovation in AI, mobile computing, and agricultural technology, AgriVision has the potential to become a cornerstone of smart farming solutions, addressing the global challenge of crop disease management and food security.

REFERENCES

1. Kulkarni, P., Karwande, A., Kolhe, T., Kamble, S., Joshi, A., & Wyawahare, M. (2021). Plant disease detection using image processing and machine learning. *arXiv preprint arXiv:2106.10698*.

2. Roy, A. M., Bose, R., & Bhaduri, J. (2022). A fast accurate fine-grain object detection model based on YOLOv4 deep neural network. *Neural Computing and Applications*, *34*(5), 3895-3921.

3. Hosen, M. D., & Islam, M. H. (2025). Aggrotech: Leveraging Deep Learning for Sustainable Tomato Disease Management. *arXiv preprint arXiv:2501.12052*.

4. El Jarroudi, M., Kouadio, L., Delfosse, P., Bock, C. H., Mahlein, A. K., Fettweis, X., ... & Hamdioui, S. (2024). Leveraging edge artificial intelligence for sustainable agriculture. *Nature Sustainability*, *7*(7), 846-854.

5. Storm, H., Seidel, S. J., Klingbeil, L., Ewert, F., Vereecken, H., Amelung, W., ... & Kuhlmann, H. (2024). Research priorities to leverage smart digital technologies for sustainable crop production. *European Journal of Agronomy*, *156*, 127178.

6. Bock, C. H., Barbedo, J. G., Del Ponte, E. M., Bohnenkamp, D., & Mahlein, A. K. (2020). From visual estimates to fully automated sensor-based measurements of plant disease severity: status and challenges for improving accuracy. *Phytopathology Research*, *2*, 1-30.

7. Schramowski, P., Stammer, W., Teso, S., Brugger, A., Herbert, F., Shao, X., ... & Kersting, K. (2020). Making deep neural networks right for the right scientific reasons by interacting with their explanations. *Nature Machine Intelligence*, *2*(8), 476-486.

8. Bohnenkamp, D., Behmann, J., & Mahlein, A. K. (2019). In-field detection of yellow rust in wheat on the ground canopy and UAV scale. *Remote Sensing*, *11*(21), 2495.

9. Mahlein, A. K., Kuska, M. T., Behmann, J., Polder, G., & Walter, A. (2018). Hyperspectral sensors and imaging technologies in phytopathology: state of the art. *Annual review of phytopathology*, *56*(1), 535-558.

10. Kuska, M. T., Brugger, A., Thomas, S., Wahabzada, M., Kersting, K., Oerke, E. C., ... & Mahlein, A. K. (2017). Spectral patterns reveal early resistance reactions of barley

against Blumeria graminis f. sp. hordei. *Phytopathology*, *107*(11), 1388-1398.

11. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in plant science*, *7*, 215232.

12. Tembhurne, J. V., Gajbhiye, S. M., Gannarpwar, V. R., Khandait, H. R., Goydani, P. R., & Diwan, T. (2023). Plant disease detection using deep learning based Mobile application. *Multimedia Tools and Applications*, *82*(18), 27365-27390.

13. Mehta, S., & Rastegari, M. (2021). Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*.

14. Wang, Y., Wang, H., & Peng, Z. (2021). Rice diseases detection and classification using attention based neural network and bayesian optimization. *Expert Systems with Applications, 178,* 114770.

15. Albahli, S., & Masood, M. (2022). Efficient attention-based CNN network (EANet) for multi-class maize crop disease classification. *Frontiers in Plant Science, 13,* 1003152.

16. Wang, G., Sun, Y., & Wang, J. (2017). Automatic image‐based plant disease severity estimation using deep learning. *Computational intelligence and neuroscience, 2017*(1), 2917536.

17. Kundu, N., Rani, G., Dhaka, V. S., Gupta, K., Nayaka, S. C., Vocaturo, E., & Zumpano, E. (2022). Disease detection, severity prediction, and crop loss estimation in MaizeCrop using deep learning. *Artificial intelligence in agriculture, 6,* 276-291.

18. Faye, D., Diop, I., Mbaye, N., Dione, D., & Diedhiou, M. M. (2023). Plant disease severity assessment based on machine learning and deep learning: A survey. *Journal of Computer and Communications, 11*(9), 57-75.

19. Jackulin, C., & Murugavalli, S. J. M. S. (2022). A comprehensive review on detection of plant disease using machine learning and deep learning approaches. *Measurement: Sensors, 24*, 100441.

20. Subbarayudu, C., & Kubendiran, M. (2024). A comprehensive survey on machine learning and deep learning techniques for crop disease prediction in smart agriculture. *Nature Environment & Pollution Technology, 23*(2).

21. Vasavi, P., Punitha, A., & Rao, T. V. N. (2022). Crop leaf disease detection and

classification using machine learning and deep learning algorithms by visual symptoms: A review. *International Journal of Electrical and Computer Engineering, 12*(2), 2079.

22. Al-Gaashani, M. S. A. M., Alkanhel, R., Ali, M. A. S., Muthanna, M. S. A., Aziz, A., & Muthanna, A. (2024). MSCPNet: A multi-scale convolutional pooling network for maize disease classification. *IEEE Access*, 1–1. https://doi.org/10.1109/ACCESS.2024.3524729

23. Zhu, D., Tan, J., Wu, C., Yung, K., & Ip, A. W. H. (2023). Crop disease identification by fusing multiscale convolution and Vision Transformer. *Sensors, 23*(13), 6015. https://doi.org/10.3390/s23136015

24. Ataman, F., & Eroglu, H. (2024). Comparative investigation of deep convolutional networks in detection of plant diseases. *Türk Doğa ve Fen Dergisi.* https://doi.org/10.46810/tdfd.1477476

25. Napoli, M. L. (2019). *Beginning Flutter: A hands-on guide to app development.* O'Reilly Media

26. Lubanovic, B. (2023). *FastAPI: Modern Python web development.* O'Reilly Media.